

7

Kafka Questions That Silence 90% of Candidates

In Backend / Senior Engineer Interviews

Save this for your next interview prep

Swipe >>

Q1 What happens when a Kafka consumer dies mid-processing?

- If auto-commit is ON: the offset may already be committed, so the message is LOST (not reprocessed).
- If auto-commit is OFF: the offset is NOT committed, so the message gets redelivered to another consumer in the group.
- This is why manual offset management + idempotent processing is critical in production systems.
- Use 'enable.auto.commit=false' and commit AFTER successful processing.

PRO TIP: This is the #1 reason for message loss in production Kafka systems.

Q2

Kafka guarantees ordering. True or False?

- PARTIALLY TRUE. Kafka guarantees ordering ONLY within a single partition, not across partitions.
- If you send messages with the same key, they land on the same partition = ordered.
- Different keys may go to different partitions = no ordering guarantee across them.
- If you need global ordering, use a single partition (but you lose parallelism).

PRO TIP: Interviewers love this trick question. Don't just say 'True'.

Q3

What is the difference between at-least-once, at-most-once, and exactly-once delivery?

- At-most-once: Messages may be lost but never redelivered. (Commit before processing)
- At-least-once: Messages are never lost but may be redelivered. (Commit after processing)
- Exactly-once: Each message processed exactly once. Requires idempotent producers + transactional APIs.
- Exactly-once has overhead. Most systems use at-least-once + idempotent consumers.

PRO TIP: Know the trade-offs. Exactly-once is NOT always the right choice.

Q4 What happens when you add a new consumer to a consumer group?

- A REBALANCE is triggered. Kafka redistributes partitions across all consumers in the group.
- During rebalance, ALL consumers in the group STOP processing temporarily.
- This causes a brief pause in consumption (can be seconds to minutes).
- Cooperative rebalancing (incremental) in newer Kafka versions reduces this downtime significantly.

PRO TIP: Mention 'CooperativeStickyAssignor' to impress the interviewer.

Q5

How does Kafka achieve high throughput?

- Sequential I/O: Kafka writes to disk sequentially (faster than random memory access).
- Zero-Copy Transfer: Data moves from disk to network without copying to application memory.
- Batching: Producers batch messages before sending, reducing network round trips.
- Page Cache: Kafka relies on OS page cache instead of JVM heap, avoiding GC pauses.

PRO TIP: This shows you understand Kafka internals, not just the API.

Q6

What is ISR in Kafka and why does it matter?

- ISR = In-Sync Replicas. It is the set of replicas that are fully caught up with the leader.
- Only ISR members can become the new leader if the current leader fails.
- If a replica falls behind (due to slow disk, network), it is removed from ISR.
- Setting 'min.insync.replicas=2' ensures at least 2 replicas confirm writes, preventing data loss.

PRO TIP: If ISR shrinks to just the leader, you risk data loss on failure.

Q7

Can Kafka be used as a database?

- Kafka is NOT a database. It is a distributed log. But it has DB-like features.
- With log compaction, Kafka retains the latest value for each key (like a key-value store).
- Kafka Streams and KTables provide stateful processing with local state stores.
- But: no indexing, no complex queries, no ACID transactions across topics.
- Use Kafka for event sourcing, not as a replacement for PostgreSQL.

PRO TIP: Mentioning log compaction + KTables shows depth of knowledge.

Found This Helpful?

- 01 **Save** this post for interview prep
- 02 **Follow** me for daily backend content
- 03 **Share** with someone preparing for interviews
- 04 **Comment** which question was new to you

Need 1:1 Interview Guidance?

Book a session on Topmate

(Link in comments)

Aman Mishra

Backend Engineer | Mentor | 7+ Years